

MYSQL

WHAT IS DATABASE?

- It is like a container where data can be collected and arranged systematically for easy management and manipulation.
- For this work we have software called DBMS (Database Management System).
- It is a software where you can Store, Retrieve, Define and Manage our data in a database.

What is RDBMS?

- Relational DBMS are mostly used DBMS models because of its easy and flexible nature.
- This model is worked on normalizing data in the database. Its gives users the ability to update, query and administer a relational database.
- SQL is typically the standard programming language used to access the database.

What and Why is SQL?

- SQL is the Standard Language for relational database management.
- You can Load, Query and Analyze massive data sets using SQL.
- Learning SQL is fun, intuitive and surprisingly easy.
- There is always need of employee that know SQL.
- Being a database + SQL expert has never ending practical benefits.

Few Common Software of database



NOTE: All are very similar to each other , based on same universal standard, with slight variations in syntax

Popular MySQL Editors



MAC, Windows Linux



Web App(Work only OS



Windows only



MAC only

Types of SQL Queries

1. DDL – Data Definition Language

It helps us to define the database structure or schema. It is used to create, modify, and delete the database structure not the data.

- List of DDL commands:

- Create - To create database or its objects
- Drop - Delete objects from database
- Alter - Alter the structure of the database
- Truncate - Remove all records from a table, including spaces
- Comment- Used to add comments
- Rename - to rename an object existing in database

Types of SQL Queries

2. DML – Data Manipulation Language

It allows us to manipulate the data present in the database instances like inserting , modifying , and deleting

- List of DML commands:

- Insert – Used to insert data into a table
- Update – Used to update existing data within a table
- Delete – Delete records from a database table
- Lock – Table control Concurrency

Types of SQL Queries

3. DQL – Data Query Language

It allows us to performing queries on the data within the schema objects. So it will fetch the data that we need to see in particular.

➤ SELECT – It is used to retrieve data from database.

4. DCL – Data Control Language

It deals with rights, permission and other controls of the database system.

➤ GRANT – Gives users access privileges to the database

➤ REVOKE – Withdraws user's access privileges.

Types of SQL Queries

5. TCL – Transaction Control Language

It deals usually with the transaction within the database.

- List of commands:

- COMMIT – Commits a Transaction or some process.
- ROLLBACK – It rollbacks a few process that done.
- SAVEPOINT – Sets a save point within a transaction.
- SET TRANSACTION – Specify characteristics for the transaction.

COMMON MySQL DATA TYPES

Data Type	Specifications
TINYINT	<i>Integer (-128 to 127)</i>
SMALLINT	<i>Integer (-32768 to 32767)</i>
MEDIUMINT	<i>Integer (-8388608 to 8388607)</i>
INT	<i>Integer (-2147483648 to 2147483647)</i>
BIGINT	<i>Integer (-9223372036854775808 to 9223372036854775807)</i>
FLOAT	<i>Decimal (precise to 23 digits)</i>
DOUBLE	<i>Decimal (23 to 53 digits)</i>
DECIMAL	<i>Decimal (to 65 digits – most precise)</i>

Data Type	Specifications
CHAR	<i>String (0 – 255)</i>
VARCHAR	<i>String (0 – 255)</i>
TINYTEXT	<i>String (0 – 255)</i>
TEXT	<i>String (0 – 65535)</i>
DATE	<i>YYYY-MM-DD</i>
DATETIME	<i>YYYY-MM-DD HH:MM:SS</i>
TIMESTAMP	<i>YYYYMMDDHHMMSS</i>
ENUM	<i>One of a number of preset options</i>

BIG 6 ELEMENTS of a SQL SELECT STATEMENT



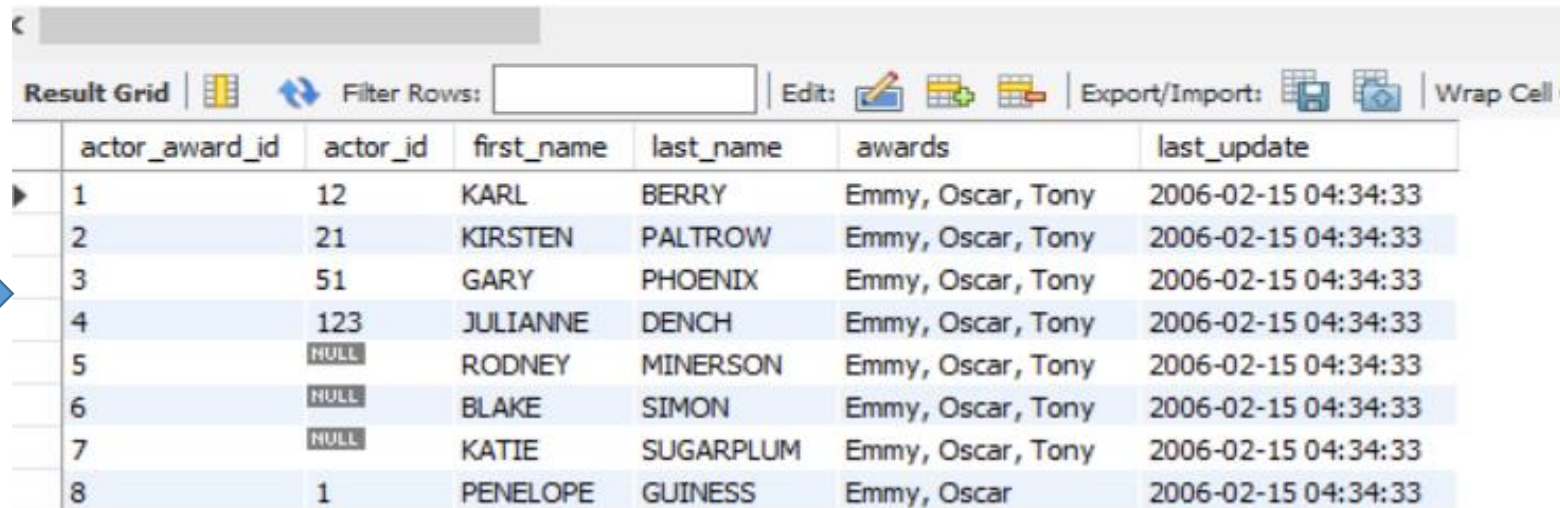
SELECT Statement

SELECT * FROM tablename

SQL all to select all columns from the specified table, using it without **WHERE clause** will return the entire table

(use semicolon after every query)

```
1 • select * from actor_award;
```



The screenshot shows a database interface with a 'Result Grid' tab selected. The grid displays 8 rows of data from the 'actor_award' table. The columns are: actor_award_id, actor_id, first_name, last_name, awards, and last_update. The data is as follows:

actor_award_id	actor_id	first_name	last_name	awards	last_update
1	12	KARL	BERRY	Emmy, Oscar, Tony	2006-02-15 04:34:33
2	21	KIRSTEN	PALTROW	Emmy, Oscar, Tony	2006-02-15 04:34:33
3	51	GARY	PHOENIX	Emmy, Oscar, Tony	2006-02-15 04:34:33
4	123	JULIANNE	DENCH	Emmy, Oscar, Tony	2006-02-15 04:34:33
5	NULL	RODNEY	MINERSON	Emmy, Oscar, Tony	2006-02-15 04:34:33
6	NULL	BLAKE	SIMON	Emmy, Oscar, Tony	2006-02-15 04:34:33
7	NULL	KATIE	SUGARPLUM	Emmy, Oscar, Tony	2006-02-15 04:34:33
8	1	PENELOPE	GUINNESS	Emmy, Oscar	2006-02-15 04:34:33

Query Result



137 16:18:16 select * from actor_award

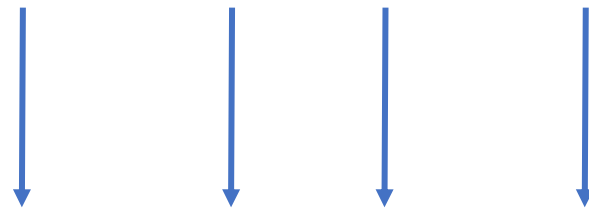
157 row(s) returned

FROM Statement

For seeing only few columns we have to write those specific column names
(use commas for after every column name except the last one)

SELECT actor_award_id,first_name,last_name,awards FROM actor_award;

4 • `select actor_award_id,first_name,last_name,awards from actor_award;`

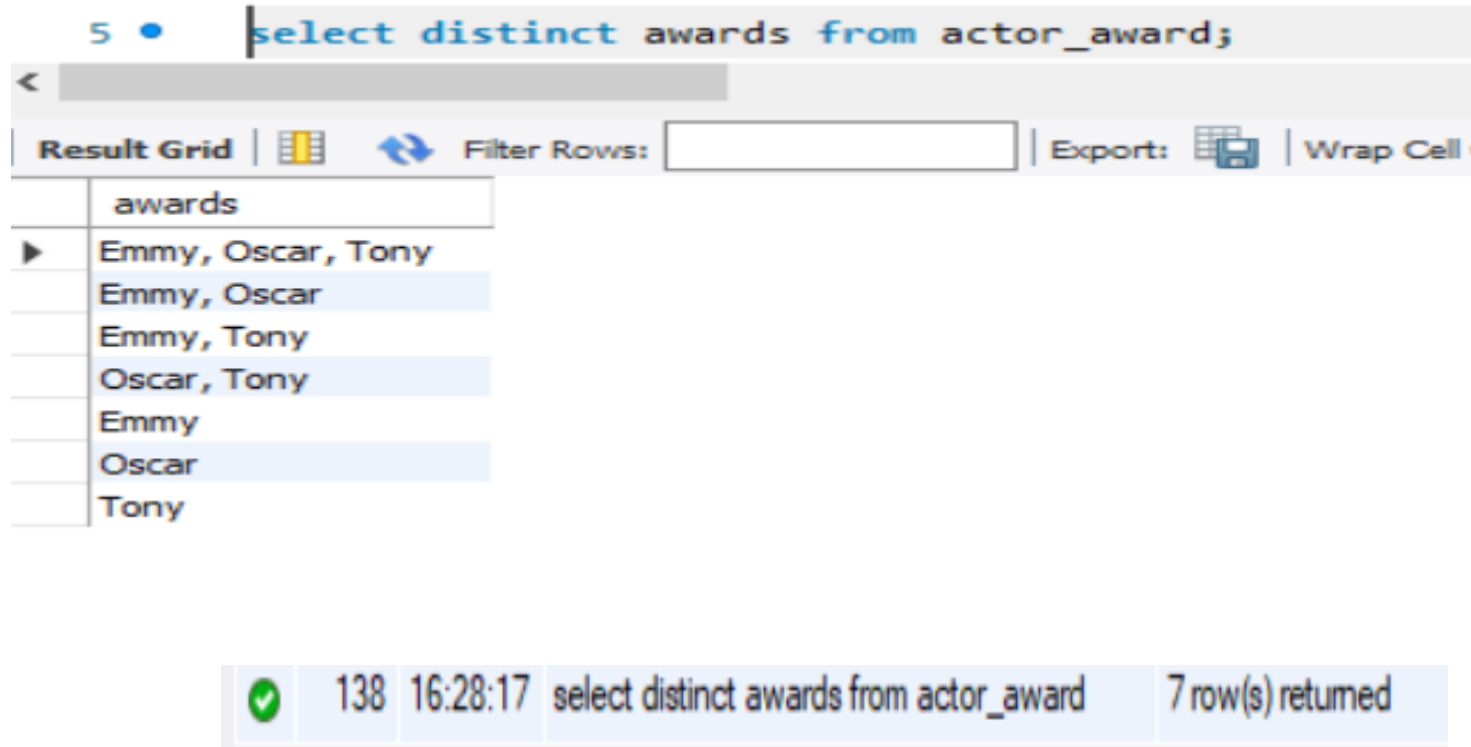


	actor_award_id	first_name	last_name	awards
1	1	KARL	BERRY	Emmy, Oscar, Tony
2	2	KIRSTEN	PALTROW	Emmy, Oscar, Tony
3	3	GARY	PHOENIX	Emmy, Oscar, Tony
4	4	JULIANNE	DENCH	Emmy, Oscar, Tony
5	5	RODNEY	MINERSON	Emmy, Oscar, Tony
6	6	BLAKE	SIMON	Emmy, Oscar, Tony
7	7	KATIE	SUGARPLUM	Emmy, Oscar, Tony

DISTINCT Statement

SELECT DISTINCT column_name from table_name;

Will only show the distinct rows from that table column.



The screenshot shows a SQL query editor with the query `select distinct awards from actor_award;` entered. Below the query, the 'Result Grid' is displayed, showing the results of the query. The results are as follows:

awards
Emmy, Oscar, Tony
Emmy, Oscar
Emmy, Tony
Oscar, Tony
Emmy
Oscar
Tony

At the bottom of the screenshot, a status bar indicates that the query was executed successfully, returning 7 rows.

✓	138	16:28:17	select distinct awards from actor_award	7 row(s) returned
---	-----	----------	---	-------------------

Only 7 rows this time because of distinct feature

WHERE statement

WHERE clause specifies criteria for filtering the records of your result set.

It can filter records using any of these logical operators. It will come after **FROM clause**. Using **AND** and **OR** we can use multiple filter at same time. Together using both clauses is a powerful.

Operator	What it Means
=	Equals
<>	Does NOT Equal
>	Greater Than
<	Less Than
>=	Greater Than Or Equal To
<=	Less Than Or Equal To
BETWEEN	A Range Between Two Values
LIKE	Matching a Pattern Like This
IN()	Equals One of These Values

Operator demo with WHERE clause

The sequence of **AND** and **OR** matters ,so use according to need.
If you are having multiple **OR** clause u can use IN.

OR rental_duration in (3,5,6)

```
3 • select * from film
4   where length > 50
5         or rental_duration = 3
6         or rental_duration = 5
7         or rental_duration = 6
8
9   and rental_rate < 3.99;
```

Even length is 48
b/c of duration 3 its
coming in result

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:									
	film_id	title	description	release_year	language_id	original_language_id	rental_duration	rental_rate	length
▶	1	ACADEMY DINOSAUR	A Epic Drama of a Feminist And a Mad Scientist ...	2006	1	NULL	6	0.99	86
	2	ACE GOLDFINGER	A Astounding Epistle of a Database Administrat...	2006	1	NULL	3	4.99	48
	4	AFFAIR PREJUDICE	A Fanciful Documentary of a Frisbee And a Lum...	2006	1	NULL	5	2.99	117
	5	AFRICAN EGG	A Fast-Paced Documentary of a Pastry Chef An...	2006	1	NULL	6	2.99	130
	6	AGENT TRUMAN	A Intrepid Panorama of a Robot And a Boy who...	2006	1	NULL	3	2.99	169
	7	AIRPLANE SIERRA	A Touching Saga of a Hunter And a Butler who ...	2006	1	NULL	6	4.99	62
	8	AIRPORT POLLOCK	A Epic Tale of a Moose And a Girl who must Con...	2006	1	NULL	6	4.99	54
	9	ALABAMA DEVIL	A Thoughtful Panorama of a Database Administ...	2006	1	NULL	3	2.99	114

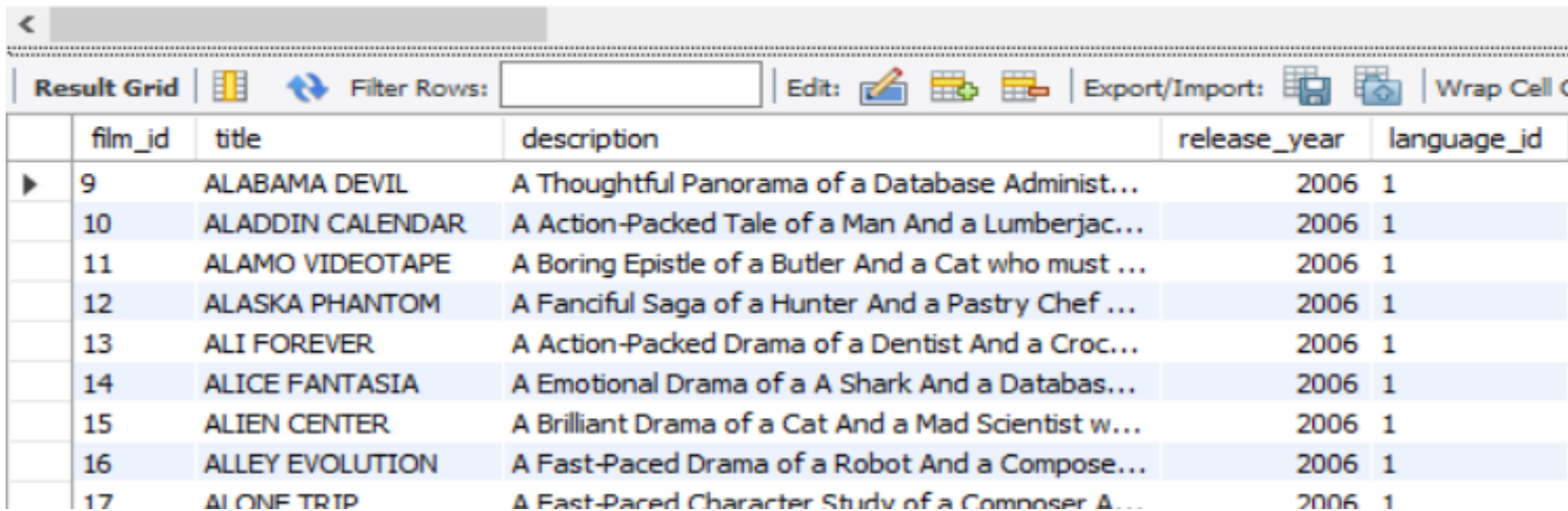
LIKE Operator

Allows you to use pattern matching in your logical operators

WILDCARD EXAMPLES	
Syntax	How SQL Evaluates Logic
column = 'value'	TRUE if the value in the column matches your value exactly (<i>nothing before or after</i>)
column LIKE 'pattern%'	TRUE if the first characters in the column match your pattern, even if they are followed by others
column LIKE '%pattern'	TRUE if the last characters in the column match your pattern, even if they are preceded by others
column LIKE '%pattern%'	TRUE if any characters in the column match your pattern, even if there are extra characters before or after
column LIKE 'pattern_'	TRUE if the first characters in the column match your pattern, even if they are followed by exactly 1 other character
column LIKE '_pattern'	TRUE if the last characters in the column match your pattern, even if they are preceded by exactly 1 other character
column LIKE '_pattern_'	TRUE if any characters in the column match your pattern, even if there is exactly 1 character before or after

LIKE operator

```
3 • select * from film
4   where title like 'AL%';
```



The screenshot shows a database query result grid. The toolbar includes a 'Result Grid' button, a 'Filter Rows' input field, and various icons for editing, exporting, and wrapping text. The table below displays the results of the query, showing film details for titles starting with 'AL'.

	film_id	title	description	release_year	language_id
▶	9	ALABAMA DEVIL	A Thoughtful Panorama of a Database Administ...	2006	1
	10	ALADDIN CALENDAR	A Action-Packed Tale of a Man And a Lumberjac...	2006	1
	11	ALAMO VIDEOTAPE	A Boring Epistle of a Butler And a Cat who must ...	2006	1
	12	ALASKA PHANTOM	A Fanciful Saga of a Hunter And a Pastry Chef ...	2006	1
	13	ALI FOREVER	A Action-Packed Drama of a Dentist And a Croc...	2006	1
	14	ALICE FANTASIA	A Emotional Drama of a A Shark And a Databas...	2006	1
	15	ALIEN CENTER	A Brilliant Drama of a Cat And a Mad Scientist w...	2006	1
	16	ALLEY EVOLUTION	A Fast-Paced Drama of a Robot And a Compose...	2006	1
	17	AL ONE TRIP	A Fast-Paced Character Study of a Composer A...	2006	1

All the title name which
start with "AL" filtered
out.

Group BY – It allows us to perform segment analysis on our data at various levels of granularity. You can use multiple columns for grouping if needed. Combining GROUP BY with aggregate functions like COUNT or SUM to specify how values are summarized for each group.

```
2 • select * from film;  
3 • select rating, rental_duration, count(film_id)  
4 from film  
5 group by rating, rental_duration;
```

Result Grid			
		Filter Rows:	
		Export:	Wrap Cell C
	rating	rental_duration	count(film_id)
▶	PG	6	39
	G	3	49
	NC-17	7	40
	G	5	33
	G	6	39
	PG	3	36
	PG-13	6	50
	R	6	27
	PG-13	3	39
	NC-17	6	57
	PG	4	41
	NC-17	5	43
	R	3	42
	R	4	53

AGGREGATE FUNCTIONS

The powerful functions below can be used GROUP BY to provide group-level summaries.

Function	Purpose
COUNT()	Count of Records <i>Skips NULL, except COUNT(*)</i>
COUNT DISTINCT()	Count of Distinct Values <i>Skips NULL values</i>
MIN()	Finds the Smallest Value <i>Skips NULL values</i>
MAX()	Finds the Largest Value <i>Skips NULL values</i>
AVG()	Average of All Values <i>Skips NULL values</i>
SUM()	SUM of All Values <i>Treats NULL values as Zero</i>

HAVING – It is an optional clause you can use with GROUP BY to limit your result set to group which certain logic criteria. It comes after GROUP BY and before ORDER BY.

```
3 • select rating, rental_duration, count(film_id) as count_id
4   from film
5   group by rating, rental_duration
6   having count_id > 30
7   and rental_duration > 3 ;
```

Alias to change column name for that query result

<			
Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
	rating	rental_duration	count_id
▶	PG	6	39
	NC-17	7	40
	G	5	33
	G	6	39
	PG-13	6	50
	NC-17	6	57
	PG	4	41
	NC-17	5	43
	R	4	53
	PG-13	5	42

The result got grouped with filter in rental_duration and count_id

Difference between WHERE and HAVING

In WHERE clause the records are filtered directly from the table while in HAVING clause the records are filtered from the groupings.

In WHERE we can't use aggregate functions to filter the records while in HAVING we can use the aggregate functions for filtering the records.

WHERE can be used with DML commands, HAVING is used with SELECT and conjunction with GROUP BY.

ORDER BY – It let you result sorted. It defaults to ascending order but can be modified to sort in descending order also.

```
3 • select rating, rental_duration, count(film_id) as count_id
4   from film
5   group by rating, rental_duration
6   Having count_id > 30
7   and rental_duration > 3
8   order by 2, 3 desc;
```

Instead of using column name we can also use column number also

Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
rating	rental_duration	count_id	
R	4	53	
PG-13	4	48	
PG	4	41	
NC-17	4	33	
NC-17	5	43	
PG-13	5	42	
R	5	40	
G	5	33	
PG	5	33	
NC-17	6	57	
PG-13	6	50	

Same query result as last one but sorted rental_duration in ascending order and count_id in descending order.

JOINS

In SQL, we use JOIN statements to do **multiple table querying**(i.e. pulling data from multiple tables at once).This can be done using the **same field**(foreign key) that exist in both the table which we are going to join.

Normalization done also using JOIN.

According to our need we will be using different types of JOIN

When you write queries using multiple tables, you need to specify both the table and column name(i.e. **inventory.inventory.id**)

COMMON JOIN TYPES

INNER JOIN

*Returns records that exist in **BOTH** tables, and excludes unmatched records from either table*

FROM leftTableName
INNER JOIN rightTableName

LEFT JOIN

*Returns ALL records from the **LEFT** table, and any matching records from the **RIGHT** table*

FROM leftTableName
LEFT JOIN rightTableName

RIGHT JOIN

*Returns ALL records from the **RIGHT** table, and any matching records from the **LEFT** table*

FROM leftTableName
RIGHT JOIN rightTableName

FULL OUTER JOIN

*Returns ALL records from **BOTH** tables, including non-matching records*

FROM leftTableName
FULL JOIN rightTableName

UNION

*Returns all data from one table, with all data from another table **appended to the end***

SELECT FROM firstName
UNION
SELECT FROM secondTableName

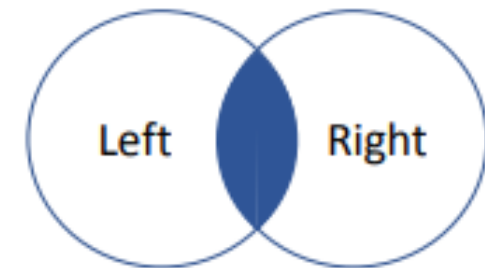
INNER JOIN

```
1 • select film.title,  
2         film.description,  
3         inventory.inventory_id,  
4         inventory.store_id  
5 from film  
6 inner join inventory  
7 on inventory.film_id = film.film_id;  
8  
9
```

This query joins the inventory table with film on film_id column as it exist in both the table.

So the column we want to get only we have to write that column name specifically.

Result Grid				
Filter Rows: Export: Wrap Cell Content: Fetch rows				
	title	description	inventory_id	store_id
▶	ACADEMY DINOSAUR	A Epic Drama of a Feminist And a Mad Scientist ...	1	1
	ACADEMY DINOSAUR	A Epic Drama of a Feminist And a Mad Scientist ...	2	1
	ACADEMY DINOSAUR	A Epic Drama of a Feminist And a Mad Scientist ...	3	1
	ACADEMY DINOSAUR	A Epic Drama of a Feminist And a Mad Scientist ...	4	1
	AFFAIR PREJUDICE	A Fanciful Documentary of a Frisbee And a Lum...	16	1
	AFFAIR PREJUDICE	A Fanciful Documentary of a Frisbee And a Lum...	17	1
	AFFAIR PREJUDICE	A Fanciful Documentary of a Frisbee And a Lum...	18	1
	AFFAIR PREJUDICE	A Fanciful Documentary of a Frisbee And a Lum...	19	1
	AGENT TRUMAN	A Intrepid Panorama of a Robot And a Boy who...	26	1
	AGENT TRUMAN	A Intrepid Panorama of a Robot And a Boy who...	27	1
	AGENT TRUMAN	A Intrepid Panorama of a Robot And a Boy who...	28	1



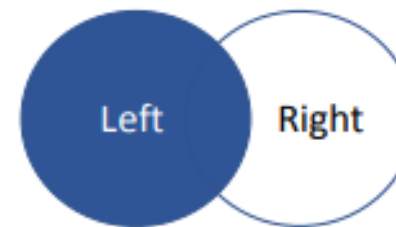
INNER JOIN on venn diagram

LEFT JOIN and RIGHT JOIN

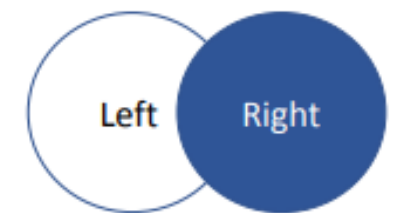
```
1 • select
2     actor.first_name,
3     actor.last_name,
4     count(film_actor.film_id) as number_of_films
5 from actor
6     left join film_actor
7         on actor.actor_id = film_actor.actor_id
8 group by actor.first_name, actor.last_name;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Cont
first_name	last_name	number_of_films	
PENELOPE	GUINNESS	19	
NICK	WAHLBERG	25	
ED	CHASE	22	
JENNIFER	DAVIS	22	
JOHNNY	LOLLOBRIGIDA	29	
BETTE	NICHOLSON	20	
GRACE	MOSTEL	30	
MATTHEW	JOHANSSON	20	
JOE	SWANK	25	
CHRISTIAN	GABLE	22	

We use LEFT JOIN here because we want to return all of the actors names , even if they don't match with any film records i.e. even there is a null value in number of films actor name will be showing not like inner join.



LEFT JOIN on venn diagram RIGHT JOIN on venn diagram

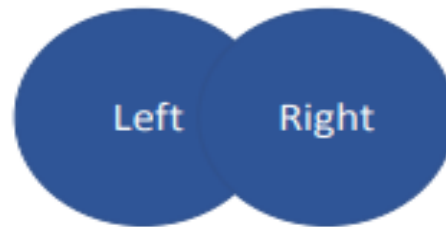


RIGHT JOIN and LEFT JOIN are just opposite to each other, It just keep all records from the second(right table).

FULL OUTER JOIN

Use a FULL JOIN when you want records from both the tables even when there isn't a match for the column you are joining on (this may yield lots of records)

It mostly come in handy if you need to merge ALL records from two tables.



FULL OUTER JOIN on venn diagram

Indexes

SQL indexes are primarily a performance tool, used to retrieve data from a database very fast. Indexing a table or view is, without a doubt, one of the best ways to improve the performance of queries and applications.

It is very useful for connecting the relational tables and searching large tables. A SQL index is a quick lookup table for finding records users need to search frequently. An index is small, fast, and optimized for quick lookups.

Indexes

SQL indexes are primarily a performance tool, used to retrieve data from a database very fast. Indexing a table or view is, without a doubt, one of the best ways to improve the performance of queries and applications.

It is very useful for connecting the relational tables and searching large tables. A SQL index is a quick lookup table for finding records users need to search frequently. An index is small, fast, and optimized for quick lookups.

Appendix

Date and Time functions

CURRENT_TIMESTAMP: Returns a datetime value containing the date and time of the computer on which the instance of SQL Server runs. The returned value does not include the time zone offset.

GETDATE : Returns a datetime value containing the date and time of the computer on which the instance of SQL Server runs. The returned value does not include the time zone offset.

GETUTCDATE - Returns a datetime value containing the date and time of the computer on which the instance of SQL Server runs. The function returns the date and time values as UTC time (Coordinated Universal Time)

DATENAME (*datepart* , *date*) - Returns a character string representing the specified *datepart* of the specified date.

DATEPART (*datepart* , *date*) - Returns an integer representing the specified *datepart* of the specified *date*.

DAY (*date*) - Returns an integer representing the day part of the specified *date*.

MONTH (*date*) - Returns an integer representing the month part of a specified *date*.

YEAR (*date*) - Returns an integer representing the year part of a specified *date*.

DATEDIFF (*datepart* , *startdate* , *enddate*) - Returns the number of date or time *datepart* boundaries, crossed between two specified dates.

DATEADD (*datepart* , *number* , *date*) - Returns a new datetime value by adding an interval to the specified *datepart* of the specified *date*.

EOMONTH (*start_date* [, *month_to_add*]) - Returns the last day of the month containing the specified date, with an optional offset.

ISDATE (*expression*) - Determines whether a datetime or smalldatetime input expression has a valid date or time value.

BRIDGING

When we need to connect two tables which do not directly relate.



Here we have no key to connect the customer table directly to city, but we can join customer to address (using `address_id`), and address to city (using `city_id`). In this case the address table serves as our bridge.

UNION

It allows to combine the result of two or more select statement into a single result set as stacking on each other results.

Each SELECT statement must include the same number of columns, Columns are in same order and stacked columns must share compatible formats

```
select
    'advisor' as type,
    first_name,
    last_name
from advisor
union
select
    'investor' as type,
    first_name,
    last_name
from investor
```

type	first_name	last_name
advisor	Barry	Beenthere
advisor	Cindy	Smartyants
advisor	Mary	Moneybags
advisor	Walter	White
investor	Montgomery	Burns
investor	Anthony	Stark
investor	William	Wonka

Above the red dotted line different query and below that is different query result

UNION VS UNION ALL

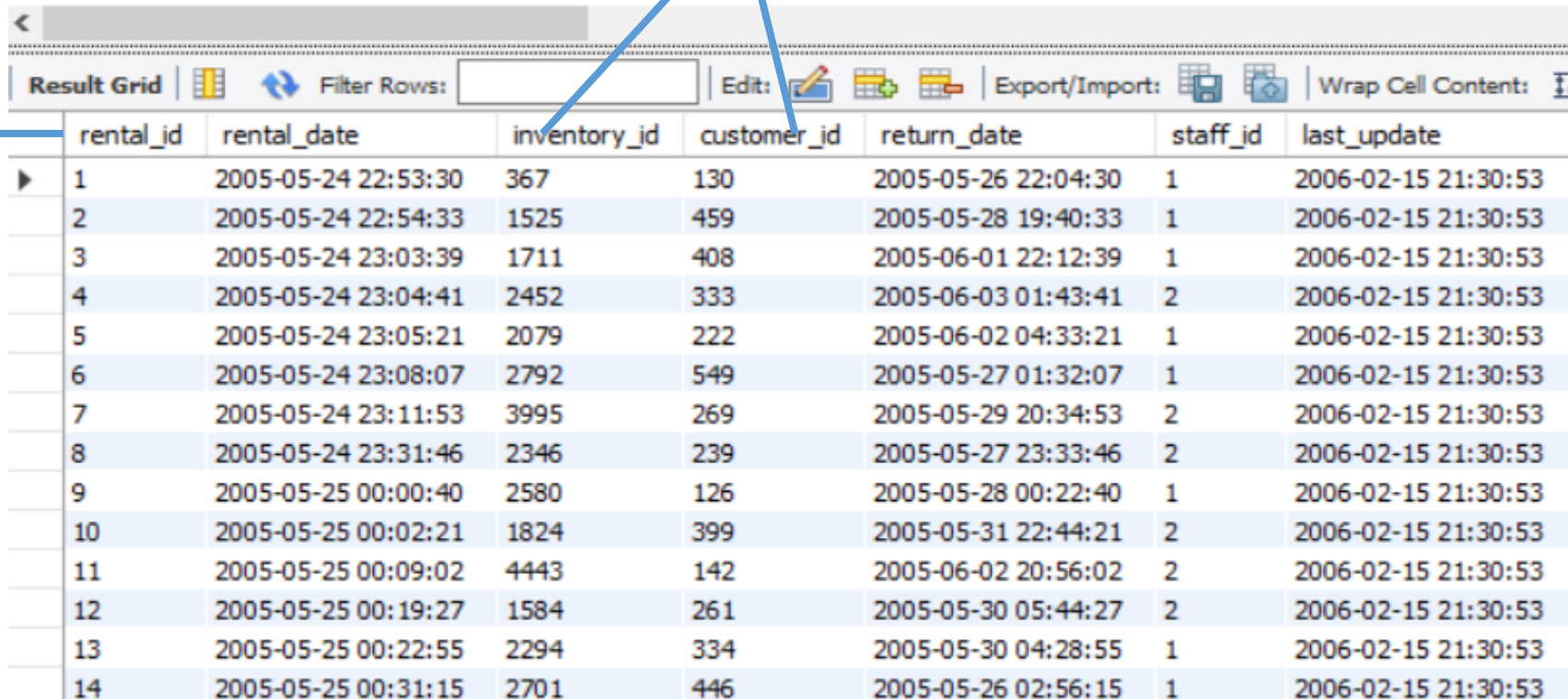
Both are almost same except just one thing that UNION doesn't allow duplicate values but union all allows duplicate value
UNION ALL most commonly used.

Primary and Foreign Key

```
3 • select * from rental;  
4
```

Foreign key for different table

Primary Key



rental_id	rental_date	inventory_id	customer_id	return_date	staff_id	last_update
1	2005-05-24 22:53:30	367	130	2005-05-26 22:04:30	1	2006-02-15 21:30:53
2	2005-05-24 22:54:33	1525	459	2005-05-28 19:40:33	1	2006-02-15 21:30:53
3	2005-05-24 23:03:39	1711	408	2005-06-01 22:12:39	1	2006-02-15 21:30:53
4	2005-05-24 23:04:41	2452	333	2005-06-03 01:43:41	2	2006-02-15 21:30:53
5	2005-05-24 23:05:21	2079	222	2005-06-02 04:33:21	1	2006-02-15 21:30:53
6	2005-05-24 23:08:07	2792	549	2005-05-27 01:32:07	1	2006-02-15 21:30:53
7	2005-05-24 23:11:53	3995	269	2005-05-29 20:34:53	2	2006-02-15 21:30:53
8	2005-05-24 23:31:46	2346	239	2005-05-27 23:33:46	2	2006-02-15 21:30:53
9	2005-05-25 00:00:40	2580	126	2005-05-28 00:22:40	1	2006-02-15 21:30:53
10	2005-05-25 00:02:21	1824	399	2005-05-31 22:44:21	2	2006-02-15 21:30:53
11	2005-05-25 00:09:02	4443	142	2005-06-02 20:56:02	2	2006-02-15 21:30:53
12	2005-05-25 00:19:27	1584	261	2005-05-30 05:44:27	2	2006-02-15 21:30:53
13	2005-05-25 00:22:55	2294	334	2005-05-30 04:28:55	1	2006-02-15 21:30:53
14	2005-05-25 00:31:15	2701	446	2005-05-26 02:56:15	1	2006-02-15 21:30:53

Primary keys cannot contain duplicates and cannot be NULL , Foreign keys can repeat and can be NULL

CONSTRAINTS

NOT NULL – Ensures that a column cannot have a NULL value

UNIQUE – Ensures that all values in a column are different

PRIMARY KEY - A combination of a NOT NULL and UNIQUE

FOREIGN KEY – Prevents actions that would destroys links between tables

CHECK – Ensures the value in column satisfies a specific condition

DEFAULT – Set a default value for a column if no value is specified

CREATE INDEX – Used to create and retrieve data from the database very quickly

PIVOTING DATA WITH COUNT & CASE

- Excel makes it easy to pivot data into columns and rows
- CASE PIVOT - Excel's ability to pivot to columns can be replicated in **SQL using COUNT and CASE**.
- When CASE Pivoting , we use COUNT() and only count records that match a certain criteria.
- Using GROUP BY to define your row labels that match a certain criteria.

```
select
    film_id,
    count(case when store_id = 1 then inventory_id else null end) as store_1_copies,
    count(case when store_id = 2 then inventory_id else null end) as store_2_copies,
    count(inventory_id) as total_copies
from inventory
group by film_id
order by film_id;
```

inventory_id	film_id	store_id	last_update
1	1	1	2006-02-15 05:09:17
2	1	1	2006-02-15 05:09:17
3	1	1	2006-02-15 05:09:17
4	1	1	2006-02-15 05:09:17
5	1	2	2006-02-15 05:09:17
6	1	2	2006-02-15 05:09:17
7	1	2	2006-02-15 05:09:17
8	1	2	2006-02-15 05:09:17
9	2	2	2006-02-15 05:09:17

Before

film_id	store_1_copies	store_2_copies	total_copies
1	4	4	8
2	0	3	3
3	0	4	4
4	4	3	7
5	0	3	3
6	3	3	6
7	2	3	5
8	0	4	4
9	3	2	5

After

Result Grid   Filter Rows:			
	customer_id	name	city
▶	1	Alice	Bengaluru
	2	Bob	Chennai
	3	Carol	Mumbai
⌵	NULL	NULL	NULL

Result Grid   Filter Rows:				
	order_id	customer_id	amount	order_date
▶	101	1	250	2025-10-10
	102	1	500	2025-10-11
	103	2	150	2025-10-12
⌵	NULL	NULL	NULL	NULL

4 -- INNER JOIN

5 **SELECT** c.name, c.city, o.amount, o.order_date

6 **FROM** customers c

7 **JOIN** orders o **ON** c.customer_id = o.customer_id;

Result Grid   Filter Rows:				
	name	city	amount	order_date
▶	Alice	Bengaluru	250	2025-10-10
	Alice	Bengaluru	500	2025-10-11
	Bob	Chennai	150	2025-10-12

Result Grid			
	customer_id	name	city
▶	1	Alice	Bengaluru
	2	Bob	Chennai
	3	Carol	Mumbai
⬇	NULL	NULL	NULL

Result Grid				
	order_id	customer_id	amount	order_date
▶	101	1	250	2025-10-10
	102	1	500	2025-10-11
	103	2	150	2025-10-12
⬇	NULL	NULL	NULL	NULL

```

10  -- LEFT JOIN
11 •  SELECT c.name, o.amount
12    FROM customers c
13   LEFT JOIN orders o ON c.customer id = o.customer id:

```

Result Grid		
	name	amount
▶	Alice	250
	Alice	500
	Bob	150
	Carol	NULL

```
10  -- RIGHT JOIN
11 • SELECT c.name, o.amount
12  FROM customers c
13  RIGHT JOIN orders o ON c.customer_id = o.customer_id;
14
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	name	amount
▶	Alice	250
	Alice	500
	Bob	150

```
10  -- LEFT JOIN
11 • SELECT c.name, o.amount
12  FROM customers c
13  LEFT JOIN orders o ON c.customer id = o.customer id;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	name	amount
▶	Alice	250
	Alice	500
	Bob	150
	Carol	NULL

```
16  -- 📌 Exercise Solution: List all customers with their total order amount
17 • SELECT c.name, SUM(o.amount) AS total_amount
18   FROM customers c
19   LEFT JOIN orders o ON c.customer_id = o.customer_id
20   GROUP BY c.name;
21
22
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	name	total_amount
▶	Alice	750
	Bob	150
	Carol	NULL

Result Grid				
	order_id	customer_id	amount	order_date
▶	101	1	250	2025-10-10
	102	1	500	2025-10-11
	103	2	150	2025-10-12
✱	NULL	NULL	NULL	NULL

24 • **select** * **from** orders

25 ✖ **CREATE INDEX** idx_orders_amount **ON** orders(amount);

26 • **SELECT** * **FROM** orders **WHERE** amount > 200;

27

Result Grid				
	order_id	customer_id	amount	order_date
▶	101	1	250	2025-10-10
	102	1	500	2025-10-11
✱	NULL	NULL	NULL	NULL

Result Grid			
	customer_id	name	city
▶	1	Alice	Bengaluru
	2	Bob	Chennai
	3	Carol	Mumbai
✱	NULL	NULL	NULL

Result Grid				
	order_id	customer_id	amount	order_date
▶	101	1	250	2025-10-10
	102	1	500	2025-10-11
	103	2	150	2025-10-12
✱	NULL	NULL	NULL	NULL

```

35 • SELECT c.name, c.city, t.total_spent
36 FROM customers c
37 JOIN (
38     SELECT customer_id, SUM(amount) AS total_spent
39     FROM orders
40     GROUP BY customer_id
41 ) t
42 ON c.customer_id = t.customer_id
43 WHERE t.total_spent > 300
44 ORDER BY t.total_spent DESC;
45

```

Result Grid			
	name	city	total_spent
▶	Alice	Bengaluru	750

Output

Available Tables

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK

Output

Available Tables

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3

